

Checkpoint1

A Teoria dos Conjuntos e Diagramas de Venn aplicados à Modelagem de Dados: Fundamentos Lógicos para Sistemas Relacionais

Tema

A Teoria dos Conjuntos e Diagramas de Venn como fundamentos para a modelagem e o tratamento de dados em Sistemas Relacionais (SQL)

Referências iniciais

[FUNDAMENTOS TEÓRICOS]:

- FERREIRA, Jaime Campos. Elementos de lógica matemática e teoria dos conjuntos. **Lisboa: Departamento de Matemática do Instituto Superior Técnico**, 2001.
[\(LINK\)](#) - Símbolos utilizados em teoria dos conjuntos e seus significados
- BAGARIA, Joan. **Set Theory**. In: ZALTA, Edward N. (ed.). The Stanford Encyclopedia of Philosophy. Stanford: Stanford University, 2023. Disponível em: <https://plato.stanford.edu/entries/set-theory/>. - Axiomas utilizados em teoria dos conjuntos
- LOUREIRO, Antonio Alfredo Ferreira. **Teoria dos Conjuntos**. Belo Horizonte: UFMG, [20--]. Disponível em: https://homepages.dcc.ufmg.br/~loureiro/md/md_5TeoriaDosConjuntos.pdf. - Introdução a teoria dos conjuntos e representação gráfica por meio de diagramas de Venn

[ALGEBRA RELACIONAL]:

- DATE, C. J. **Introdução a Sistemas de Bancos de Dados**. Rio de Janeiro: Elsevier, 2004. [\(link\)](#) - Trabalho robusto que aborda fundamentos teóricos básicos, álgebra relacional e bancos de dados relacionais. Cap 7 introduz os conceito de álgebra relacional abordados no artigo
- HEUSER, Carlos Alberto. **Projeto de Banco de Dados**. 6. ed. Porto Alegre: Bookman, 2009. [\(link\)](#) - Atentar-se ao capítulo 1 e 4, em que fundamentos de bancos de dados relacionais são estabelecidos a partir da álgebra relacional.
- CODD, E. F. A Relational Model of Data for Large Shared Data Banks. Communications of the ACM, v. 13, n. 6, p. 377–387, 1970.. [\(link\)](#) - Foco em algebra relacional nas seções 1.3, e 2

[APLICAÇÃO TÉCNICA]:

- DATE, C. J. SQL e Teoria Relacional: como escrever código SQL preciso. Tradução de Edson Furmankiewicz. São Paulo: Novatec, 2012. [\(link\)](#) - Conceito básicos de SQL, sintaxe, uso e estrutura de dados
- ATWOOD, Jeff. A Visual Explanation of SQL Joins. Coding Horror, 2007. Disponível em: <https://blog.codinghorror.com/a-visual-explanation-of-sql-joins/>. - Explicação didática de como representações em SQL facilitam a compreensão humana de relação de conjuntos
- BEAULIEU, Alan. Learning SQL: Generate, Manipulate, and Retrieve Data. 3. ed. Sebastopol: O'Reilly Media, 2020. [\(link\)](#) - Ponte que explica SQL e a construção de bancos de dados relacionais partindo de conhecimentos de álgebra relacional. Conectando o conhecimento teórico com a aplicação técnica
- STONEBRAKER, Michael; HELLERSTEIN, Joseph M. What Goes Around Comes Around. **Readings in Database Systems**, 4. ed., MIT Press, 2005. [\(link\)](#) - Traz a forma com que o SQL foi um marco para a computação ao corrigir problemas de falta de padronização de dados e seu armazenamento
- CHAMBERLIN, Donald D. Early History of SQL. IEEE Annals of the History of Computing, v. 34, n. 4, p. 78-82, out./dez. 2012. [\(link\)](#) importância e vantagens do uso de Sistemas Relacionais para armazenar, processar e avaliar dados
- WINAND, Markus. **SQL Performance Explained**: everything developers need to know about SQL performance. [S. l.]: Markus Winand, 2012. [\(link\)](#)
- SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. **Query Optimization**. [S. l.: s. n.], [20--]. (Slides de apoio ao livro Database System Concepts). Disponível em: <https://www.db-book.com/slides-dir/PDF-dir/ch16.pdf>.
- RAHM, Erhard; DO, Hong Hai. **Data Cleaning: Problems and Current Approaches**. IEEE Bulletin of the Technical Committee on Data Engineering, [s. l.], v. 23, n. 4, p. 3-13, 2000. [link](#)
- KENT, William. A Simple Guide to Five Normal Forms in Relational Database Theory. **Communications of the ACM**, New York, v. 26, n. 2, p. 120-125, fev. 1983. [link](#)

Problema: De que maneira a negligência dos fundamentos da Teoria dos Conjuntos e das Formas Normais, durante a etapa de modelagem lógica, compromete a integridade dos dados e a confiabilidade da informação gerada por sistemas de bancos de dados relacionais? **Problema é citado nas seguintes fontes:** CODD (1970) Seção 1.2.1; DATE (2012): No Capítulo 1 ("Logic and Databases"); KENT (1983) "Anomalies" (nas páginas 120-122); RAHM & DO (2000) Seção 2.1 ("Data Cleaning Problems"). ATWOOD (2007) Joins are the bread and butter of SQL;

Objetivo principal

O objetivo do projeto é identificar de que maneira a aplicação dos fundamentos da Teoria dos Conjuntos e das Formas Normais, na etapa de modelagem de dados, influencia a integridade e a confiabilidade da informação em bancos de dados relacionais. Como objetivo, tem-se que: **Objetivos similares são citados como soluções nas seguintes fontes CODD (1970) Seção 1.3 (A Relational Model of Data); KENT (1983) Seção "Objectives of Normalization" (pág. 120); DATE (2012) Capítulo 2 (The Relational Model) e Capítulo 3 (The Anatomy of a Relation); ATWOOD (2007) "A Visual Explanation of SQL Joins"; LOUREIRO ([20--]) Seção 5 (Representação Gráfica).**

Objetivos

1. Demonstrar a eficácia do uso de Diagramas de Venn como instrumento de validação lógica e visual, assegurando a conformidade das relações e a compreensão humana das informações exibidas.
2. identificar e corrigir as anomalias de integridade e inconsistências informacionais decorrentes da negligência dos preceitos teóricos na modelagem de dados;
3. Entender como a negligência das Formas Normais impactam na estrutura do banco de dados (onde a negligência de cada norma pode ser evidenciada e o que ela acarreta)

Justificativa

#Reconhecimento da literatura prévia

Estudos clássicos e contemporâneos demonstram que a integridade dos dados é o pilar fundamental para o sucesso operacional (Codd, 1970), a precisão em sistemas de suporte à decisão (Date, 2012) e a confiabilidade da informação organizacional (Stonebraker & Hellerstein, 2005). Comumente, a literatura apresenta que falhas na estruturação lógica estão associadas a inconsistentes arquiteturas de informação e processos de tratamento de dados onerosos (Rahm & Do, 2000), exigindo a adoção de rigor matemático e normalização precoces.

#Salientar, novamente, o problema

Diante desse cenário, um dos desafios fundamentais dos gestores e engenheiros de dados é garantir a integridade referencial e a veracidade da informação na origem, mitigando anomalias que comprometem a confiança nos ativos digitais da companhia.

#Destacar o diferencial deste projeto

Embora seja elevada a quantidade de manuais técnicos que tratam sobre a sintaxe da linguagem SQL (Beaulieu, 2020), ainda não há consenso ou profundidade acadêmica sobre o impacto direto da negligência teórica da Álgebra Relacional na confiabilidade final do dado. Na literatura, encontra-se o tema sendo abordado de forma fragmentada, focando em aspectos técnicos isolados, tais como: otimização de consultas (Silberschatz et al., [20--]); processos de limpeza e *data cleaning* (Rahm & Do, 2000); e regras formais de normalização (Kent, 1983).

#Benefício em ler/produzir este artigo

Neste escopo, evidencia-se uma necessidade de que os estudos sobre bancos de dados mudem seu enfoque de uma abordagem acentuadamente específica para uma orientação baseada em conceitos lógicos de conjuntos, que possam ser, de forma satisfatória, apresentados, preferencialmente, por meio de Diagramas de Venn e em compreender os fatores que levam, em primeira instância, o modelador de dados do Sistema Relacional a negligenciar convenções da teoria de conjuntos e das Formas Normais

Checkpoint 2

Revisão Conceitual e Definições formais centrais

Pilar 1 - Conjuntos

- Definição de conjuntos e pertencimento
- Símbolos de conjuntos ($\mathbb{Z}$, $\mathbb{I}$, $\mathbb{R}$...)
- Operações com conjuntos(União, diferença, interseção)
- Relação de conjuntos(Contém, está contido)
- Representação em diagramas de venn

Pilar 2 - Diagramas de Venn

Pilar 3 - Algebra Relacional / Dados

- Definir diferentes estruturas de dados (tuplas,relações/tabelas, dicionários, atributo e domínio)
- Diferentes formas de salvar dados em arquivos (JSON, CSV, XML)
- Operações de filtrar e selecionar linhas em SQL (Where, Select)
- Combinação de tabelas (União, Interseção e Diferença)
- Explicar chaves estrangeiras e chaves primárias

Pilar 4- Formas Normais

- Normalização da representação de conjunto de dados
- 3 Formas Normais
- Definir as principais anomalias de dados

Pilar 5 - Bancos de dados

- Banco de dados relacional
- SQL

Revisão Conceitual

O que é -> Por que -> De acordo com quem -> Explicação com texto

Definições Formais Centrais

Axiomas -> Símbolos -> Estritamente técnico e objetivo -> Matemática

SEÇÕES DE TRABALHO

1° - Matemática e Diagramas de Venn:

Explicar como trabalhar com dados envolve a teoria dos conjuntos e diagramas de venn -> expor operações em conjuntos (união interseção) podem ser feitos com diagramas de venn e utilizando a linguagem matemática. O Foco desta parte é começar com o simples, e mostrar como a matemática pode chegar até bancos de dados mais complexos

2° - Algebra Relacional:

Traduzir a seção 1 para um modelo mais simples de se entender -> Detalhar os tipos de dados (dicionários, tabelas, listas, tuplas) e fazer a ponte da linguagem matemática até

esse tipo de representação. Falar sobre as operações de filtrar e selecionar (traduzir da seção 1); Falar sobre chaves estrangeiras e primárias (traduzir da seção 1)

3° - Formas Normais:

A partir da seção 1 e 2, explicar como isso resulta nas 3 Formas Normais.

Explicar como as formas normais limpam e padronizam a representação de dados.

Detalhar todas as anomalias de dados e explicar como elas são resultado da negligência das formas normais

4° - Casos

Mostrar 1 caso de banco de dados com problemas, cada um com um problema resolvida por uma FN-> detalhar cada norma padrão negligenciada e como isso impacta a integridade dos dados -> Solucionar os problemas, passo a passo, com o as Formas Normais e no final fazer uma representação do banco (antes e depois da solução do problema) em diagramas de Venn para provar que ele pode ser uma ferramenta útil para validação do banco de dados

5° - Estruturas relacionais + Venn

Escolher algum software (PostgreSQL , MySQL , SQL Server) e falar sobre como eles surgiram de forma breve, quais suas vantagens por ser um banco de dados relacional (falar sobre isso com base na seção 2)

Mostrar que as funções em SQL(escolher mais de 3) são fruto de conceitos matemáticos mostrados na seção 1 e 2 e podem ser ilustradas com diagramas de Venn

Expor um cenário de um banco de dados relacional bem modelado e apontar as Formas Normais utilizadas em cada etapa(seção 3), onde a álgebra relacional aparece(seção 2) e como representar ele com diagramas de venn (seção 1)

Revisão Conceitual

Pilar 1-Conjuntos

De acordo com Antonio Alfredo Ferreira Loureiro, a Teoria dos Conjuntos é o ramo da matemática que estuda as propriedades e as relações entre coleções de objetos. Um conjunto é entendido como uma coleção de objetos bem definidos, que podem ser qualquer coisa: números, letras, pessoas ou até outros conjuntos e são denominados elementos.

A relação de pertencimento é o conceito fundamental dessa teoria. Segundo Jaime Campos Ferreira, se um objeto x é um dos elementos que compõem o conjunto A , dizemos que x pertence a A , utilizando o símbolo $\in$ ($x \in A$). Caso o objeto não faça parte da coleção, utiliza-se o símbolo $\notin$ ($x \notin A$).

Ferreira reforça que a relação de pertencimento é a base para definir outras relações, como a de inclusão (quando um conjunto está contido em outro).

Na prática, se definirmos um conjunto A como os números pares menores que 10, temos $A = \{2, 4, 6, 8\}$.

- Dizemos que $2 \in A$ (2 pertence a A), pois é um elemento do conjunto
- Dizemos que $3 \notin A$ (3 não pertence a A), pois não é um elemento do conjunto

Essa distinção clara entre o que está "dentro" e o que está "fora" do conjunto, conforme as regras de lógica matemática detalhadas por Ferreira, é o que permite operações como a seleção, a filtragem e a validação de dados em bancos de dados.

A necessidade de definir conjuntos e pertencimento reside na fundamentação de toda a matemática moderna. Joan Bagaria explica que a Teoria dos Conjuntos serve como a linguagem comum e a fundação sobre a qual quase todos os objetos matemáticos (como números, funções e estruturas geométricas) são formalmente definidos. Além disso, Loureiro destaca que essa estrutura permite a organização de informações e a visualização de relações complexas através de métodos gráficos, como os Diagramas de Venn, facilitando a compreensão de interseções e uniões entre diferentes grupos.

A definição moderna de conjunto vai além da simples intuição. Como aponta Bagaria, para evitar contradições lógicas, a matemática utiliza axiomas (regras fundamentais). Um dos mais importantes é o Axioma da Extensionalidade, que afirma que um conjunto é determinado unicamente pelos seus elementos, ou seja, a ordem em que os elementos são apresentados ou a repetição deles não altera a identidade do conjunto.

Pilar 2 - Diagramas de Venn

Venn (1880), em *On the Diagrammatic and Mechanical Representation of Propositions and Reasonings*, propõe os diagramas como um instrumento de verificação lógica das relações entre conjuntos, permitindo testar a validade de proposições de forma sistemática e rigorosa.

No escopo do uso de diagramas como instrumentos de verificação das relações entre conjuntos, faz-se imprescindível salientar a diferença entre Diagramas de Venn e Diagramas de Euler. Moktefi e Lemanski (2022), em *On the Origin of Venn Diagrams*, formalizam a propriedade que os distingue: os Diagramas de Venn representam todas as regiões possíveis resultantes da combinação de conjuntos, inclusive as regiões vazias, as quais são marcadas por sombreamento. Por sua vez, os Diagramas de Euler representam apenas as relações efetivamente existentes. Um diagrama que não representa áreas sem elementos não permite verificar se uma operação retorna um conjunto vazio, o que é uma das verificações necessárias para avaliar a precisão dos resultados em consultas SQL. A título de exemplificação, um Diagrama de Venn de dois conjuntos, A e B , possui exatamente quatro regiões: $A \setminus B$ (exclusiva de A), $A \cap B$ (termos em comum ou superposição), $B \setminus A$ (exclusiva de B) e $(A \cup B)^c$ (o complemento da união entre A e B , ou seja, região externa aos conjuntos A e B). Cada operação fundamental corresponde a uma combinação específica dessas regiões, o que torna o diagrama um mapa completo das possíveis relações entre os dois conjuntos.

Pilar 3 - Álgebra Relacional

O Modelo Relacional, proposto por Edgar F. Codd em 1970 no artigo *A Relational Model of Data for Large Shared Data Banks*, conecta os fundamentos da Teoria dos Conjuntos ao domínio computacional de dados. Na seção 1.1 do artigo, Codd define: dada uma coleção de domínios $D_1, D_2, \dots, D_n$, uma relação R sobre esses domínios é um subconjunto do produto cartesiano $D_1 \times D_2 \times \dots \times D_n$. Nesse modelo, cada relação é composta por atributos (colunas) e tuplas (linhas), cujos valores pertencem a domínios específicos. Essa é

a mesma definição matemática de relação formulada por Enderton (1977). Na seção 2.1, Codd define as operações relacionais, como seleção, projeção, união, interseção e diferença, como operações de conjuntos sobre os subconjuntos do produto cartesiano. Dentre essas operações, a seleção é responsável pelo filtro de tuplas (linhas), enquanto a projeção seleciona atributos específicos. A título de exemplificação, em um conjunto de dados contendo os atributos nome e nota, a seleção poderia ser utilizada para retornar apenas as tuplas cujas notas sejam superiores a cinco. Em contrapartida, a projeção selecionaria apenas atributos específicos, como a coluna correspondente aos nomes. Sob esse prisma, evidencia-se que o Modelo Relacional consolida a relação entre a Teoria de Conjuntos e o domínio de dados.

Sob o diapasão das inovações que Codd introduziu, a exigência de que a união de duas relações só é definida se elas possuírem o mesmo número de atributos com domínios correspondentes compatíveis, pois apenas assim o resultado constitui uma relação válida com esquema consistente, uma decisão de modelagem, não uma propriedade da Teoria dos Conjuntos, e demonstra que a transposição de Codd foi deliberada e tecnicamente informada. Stonebraker e Hellerstein (2005), [em What Goes Around Comes Around](#), documentam que a vantagem do Modelo Relacional sobre os sistemas hierárquicos se deveu à sua base conjuntista rigorosa. Sistemas que careciam de fundamentos matemáticos bem definidos não conseguiam garantir que a mesma consulta produzisse sempre o mesmo resultado, nem que operações equivalentes fossem reconhecidas como idênticas. Nesse contexto, a base conjuntista resolveu esses problemas, tornando as operações sobre dados baseadas na Teoria dos Conjuntos, tornando-as previsíveis e independentes de implementação.

Silberschatz, Korth e Sudarshan (2019), [em Database System Concepts \(7ª ed.\)](#), o [livro-texto mais adotado em graduações de Computação no mundo](#), descrevem a Álgebra Relacional como extensão da Teoria dos Conjuntos e estabelecem a correspondência entre operações relacionais e SQL. Date (2004), [em Introdução a Sistemas de Bancos de Dados \(8ª ed.\)](#), descreve cada operação da Álgebra Relacional como operação de conjuntos, incluindo a união, interseção, diferença, produto cartesiano, seleção e projeção. Date também [em SQL e Teoria Relacional](#), demonstra que a precisão de consultas SQL exige o domínio da teoria relacional e, por extensão, das operações de conjuntos que a fundamentam. Heuser (2009), [em Projeto de Banco de Dados \(6ª ed.\)](#), identifica que erros recorrentes em projetos de banco de dados têm origem na incompreensão das relações lógicas entre os conjuntos de dados que as tabelas representam.

Pilar 4 - Bancos de dados

Um banco de dados é uma coleção organizada de dados persistentes que atende a múltiplos usuários e aplicações. O modelo de banco de dados relacional fundamenta-se na formalização proposta por Codd, segundo a qual os dados devem ser organizados em relações compostas por tuplas e atributos (Codd, 1970), ou seja, no modelo relacional, os dados são organizados em tabelas com esquema definido, sendo pouco flexível, mas muito organizado, íntegro e de fácil consulta, infere-se, portanto, que sua estrutura não é

aleatória, mas planejada de acordo com as necessidades de armazenamento, organização e recuperação das informações. Nesse sentido, em um banco de dados relacional, uma tabela, por exemplo, não constitui apenas um mecanismo prático de armazenamento, mas uma representação formal de um conjunto matemático, em que cada tupla corresponde a um elemento pertencente a um domínio definido. A adoção desse modelo está associada à necessidade de reduzir redundâncias e inconsistências, problemas recorrentes em sistemas que não utilizam uma estrutura lógica rigorosa (Heuser, Projeto de Banco de Dados). Conforme observado por Rahm e Do, a ausência de organização adequada dos dados implica em custos elevados de correção e manutenção, comprometendo a confiabilidade das informações (Rahm & Do, Data Cleaning). Do ponto de vista teórico, o modelo relacional é uma aplicação direta da teoria dos conjuntos e da lógica de predicados, sendo essas bases essenciais para a consistência estrutural dos dados (Date, SQL and Relational Theory). A operacionalização desses conceitos ocorre por meio da linguagem SQL, desenvolvida no contexto do System R, que traduz operações matemáticas em comandos declarativos acessíveis ao usuário (Chamberlin, Early History of SQL).

Definições Formais Centrais

Pilar 1-Conjuntos

Símbolos de Conjuntos Numéricos:

- $\mathbb{N}$: Conjunto dos números **Naturais** $\{0, 1, 2, 3, \dots\}$.
- $\mathbb{Z}$: Conjunto dos números **Inteiros** $\{\dots, -2, -1, 0, 1, 2, \dots\}$.
- $\mathbb{Q}$: Conjunto dos números **Racionais** (definidos por frações $\frac{a}{b}$).
- $\mathbb{I}$ ou $\mathbb{R} \setminus \mathbb{Q}$: Conjunto dos números **Irracionais** (decimais infinitos não periódicos).
- $\mathbb{R}$: Conjunto dos números **Reais** (união dos racionais com irracionais).
- $\mathbb{C}$: Conjunto dos números **Complexos**.
- U : **Conjunto Universo**

Operações com Conjuntos:

- **União** ($\cup$): $A \cup B = \{x | x \in A \vee x \in B\}$
- **Interseção** ($\cap$): $A \cap B = \{x | x \in A \wedge x \in B\}$
- **Diferença** ($-$ ou $\setminus$): $A - B = \{x | x \in A \wedge x \notin B\}$
- **Complementar** (A^c ou $\bar{A}$) = $\{x \in U | x \notin A\}$

Propriedades de Operações com Conjuntos:

- $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$
- $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$
- $(A \cup B)^c = A^c \cap B^c$
- $(A \cap B)^c = A^c \cup B^c$

Relação de Conjuntos (Inclusão):

- **Está contido** ($\subset$ ou $\subseteq$): $A \subset B \Leftrightarrow \forall x (x \in A \Rightarrow x \in B)$
- **Contém** ($\supset$ ou $\supseteq$): $B \supset A$. Indica que B possui todos os elementos de A .
- **Não está contido** ($\not\subset$) = $\exists x (x \in A \wedge x \notin B)$
- **Conjunto Vazio** ($\emptyset$) = $\{x \mid x \neq x\}$.

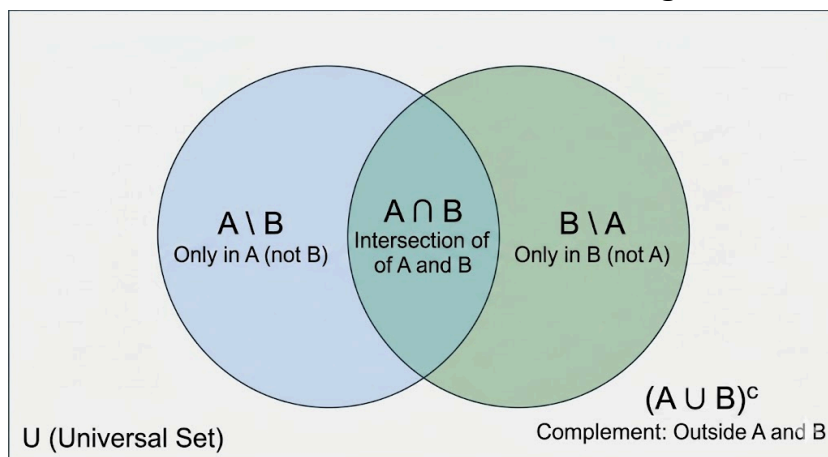
Fundamentação Axiomática (Estritamente Técnico):

• **Axioma da Extensionalidade:** $B \Leftrightarrow \forall x (x \in A \leftrightarrow x \in B)$. Define que dois conjuntos são iguais se, e somente se, possuem os mesmos elementos. É a base para as relações de igualdade e inclusão.

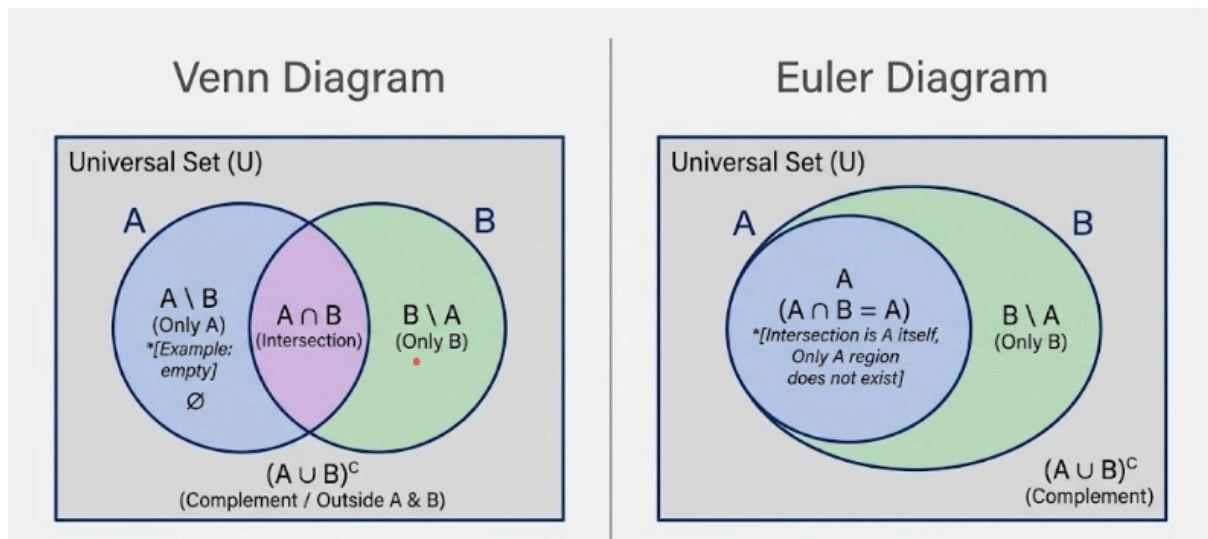
• **Axioma do Par / União:** $\forall A \forall B \exists C \forall x (x \in C \leftrightarrow (x \in A \vee x \in B))$. Garante que, dados dois conjuntos, a união deles também resulta em um conjunto válido dentro do sistema matemático.

Produto Cartesiano: Dados dois conjuntos A e B , o produto cartesiano de A e B é o conjunto: $A \times B = \{(a, b) \mid a \in A \wedge b \in B\}$ O produto cartesiano é o conjunto de todos os pares ordenados (a, b) com a em A e b em B . Esta é a definição que Codd (1970, seção 1.1) transpõe diretamente para definir relações de banco de dados.

Pilar 2 - Diagramas de Venn



- $R1 = A \setminus B = \{x \mid x \in A \wedge x \notin B\}$
- $R2 = A \cap B = \{x \mid x \in A \wedge x \in B\}$
- $R3 = B \setminus A = \{x \mid x \notin A \wedge x \in B\}$
- $R4 = (A \cup B)^c = \{x \in U \mid x \notin A \wedge x \notin B\}$



Pilar 3 - Álgebra relacional

Estruturas relacionais:

- **Relação** subconjunto do produto cartesiano entre diferentes domínios $D_1, D_2, \dots$, formalmente representado por:
 $R \subseteq D_1 \times D_2 \dots$, ou seja,
- **Tupla** elemento pertencente a uma relação, R , ou seja, $t \in R$. Em bancos de dados relacionais, são representadas pelas linhas
- **Atributo** descrição dos dados armazenados em uma relação. É associado a um domínio específico, restringindo o tipo de valores que podem ser, em um contexto tabular, armazenados em cada coluna
- **Domínio** Conjunto de valores válidos que podem ser armazenados em um atributo.

Operações relacionais:

- **Seleção** operação que, por meio de uma condição, denominada de filtro, filtra tuplas com base em um predicado booleano. Define-se formalmente como:
 - $\sigma_{\phi}(R) = \{ t \mid t \in R \wedge \phi(t) \}$, onde R é a relação original, t é uma tupla (linha da tabela original) que pertença a R e que satisfaça a condição imposta.
- **Projeção** operação que isola um subconjunto de atributos. Define-se formalmente como:

- $\pi_L(R) = \{ t[L] \mid t \in R \}$, onde L são os atributos (colunas) que queremos selecionar da relação, R , original e $t[L]$ é a tupla do atributo selecionado.
- **Produto Cartesiano** A combinação de todas as tuplas de uma relação com todas as tuplas de outra, em que o resultado é uma nova relação que contém todas as combinações possíveis entre as tuplas das relações originais. Formalmente representado por:
 - $R \times S = \{ (r,s) \mid r \in R \wedge s \in S \}$
- **Junção** resultado do produto cartesiano entre duas relações com um filtro aplicado (σ), ou seja, $\sigma_\phi(R \times S) = (R \bowtie_\theta S)$, ou sem um filtro, mas com a substituição de lacunas por algum valor.

Pilar 4 - Bancos de dados

- **Tabelas** no paradigma relacional, representam logicamente uma relação R , onde R é um subconjunto do produto cartesiano entre os domínios. Consequentemente, a ordem das linhas é irrelevante e, em tese, os elementos não são duplicados.
- **Colunas** representam os atributos de uma tabela (relação), possuindo um domínio definido que delimita os valores válidos que podem ser armazenados. Possuem nome, que é apresentado em um cabeçalho.
- **Linguagem SQL** linguagem padrão para o modelo relacional, que traduz a álgebra e o cálculo relacional em comandos computacionais, permitindo a definição, manipulação e consulta dos conjuntos de dados.
- **Chave Primária (PK)** é um atributo que dá, a uma tupla, uma identificação única, garantindo a integridade da relação. Formalmente exemplificada em: $t_1, t_2 \in R$, se $t_1 \neq t_2$, então $t_1[PK] \neq t_2[PK]$.
- **Chave Estrangeira (FK)** atributo que referencia a PK em uma segunda relação, garantindo a integridade referencial em uma tabela distinta da original, permitindo maior flexibilidade das estruturas de dados. Para que FK exista em R_2 , ele, necessariamente, deve existir em R_1 .
- **SELECT:** implementa a operação de projeção (π)
- **WHERE:** implementa a operação de seleção (σ)
- **UNION:** Realiza a união de conjuntos ($\cup$) entre duas relações, em que os domínios sejam compatíveis
- **JOIN:** Combina tuplas de relações distintas através de atributos comuns.
 - **INNER:** Retorna $(R \bowtie_\theta S)$
 - **OUTER:** Retorna $(R \bowtie S)$: e adiciona valores nulos no local de lacunas
 - **LEFT** Retorna todas as tuplas de R (está à esquerda). Se não houver correspondência em S , os atributos recebem NULL.
 - **RIGHT** Retorna todas as tuplas de S (está à direita). Se não houver correspondência em R , os atributos recebem NULL.

- **FULL** Retorna todas as tuplas de R e S , aplicando NULL onde a correspondência for inexistente.

Introdução

A Teoria dos Conjuntos e Diagramas de Venn aplicados à Modelagem de Dados: Fundamentos Lógicos para Sistemas Relacionais

Marco Antônio Rodrigues Siqueira

Gael Ceregatti Murad

Davi Brandão

Carlos Marquim

Lucas Viana

Rafael Dias

Resumo: Na contemporaneidade, a relevância das empresas é diretamente proporcional ao gerenciamento dos dados referentes às suas operações. Adjacente à popularização da ciência de dados, linguagens voltadas à estruturação, manutenção e visualização de dados, como o SQL, foram desenvolvidas com o propósito de democratizar o acesso às vantagens proporcionadas pela análise de grandes volumes de dados. Entretanto, o foco restrito à parte prática da análise em detrimento do ferramental teórico, fundamentado na Teoria dos Conjuntos e na Álgebra Relacional, gera, até os dias de hoje, problemas envolvendo redundâncias em bancos de dados extensos, comprometendo, portanto, a veracidade e a acurácia das informações extraídas a partir de sua coleta e análise. Este trabalho analisa as conexões formais entre o Modelo Relacional e a Teoria dos Conjuntos, e propõe a validação lógica por meio do uso de Diagramas de Venn, por serem inteligíveis tanto para leigos quanto para profissionais, e por sua capacidade de mapear todas as regiões e combinações possíveis de uma operação, constituindo uma eficaz ferramenta de revisão humana e verificação rigorosa que garante a integridade das consultas e da estrutura lógica dos sistemas relacionais.

Palavras-chave: Banco de Dados Relacional; SQL; Teoria dos Conjuntos; Álgebra Relacional; Diagramas de Venn; Redundância de Dados.

Introdução

A hiperconectividade e a digitalização são pilares fulcrais e preponderantes no mercado de trabalho hodierno (Stonebraker & Hellerstein, 2005). Percebe-se que, quanto mais digitalizado, maior a exigência por sistemas capazes de armazenar, processar e avaliar dados (Chamberlin, 2012), com o mote de que estes sejam utilizados como ativos estratégicos para otimizar vendas, administrar negócios e organizar as dinâmicas laborais (Date, 2012). Essa estruturação corrobora a precisão em sistemas de suporte à decisão; característica que (Codd, 1970) é determinante para o desempenho de uma empresa no século XXI.

Atualmente, a literatura moderna indica que organizações com uma estrutura sólida de armazenamento e análise de dados são mais capacitadas para alocar recursos em áreas estratégicas e mobilizar modelos de negócio inovadores, atingindo uma performance satisfatória e livre de inconsistências (Codd, 1970; Stonebraker & Hellerstein, 2005). Neste escopo, a vasta gama de possibilidades competitivas ressalta a importância de adquirir um sistema de banco de dados que seja não apenas robusto em capacidade, mas também intuitivo para que profissionais de diferentes setores possam manipular informações conforme as demandas do negócio (Date, 2012; Beaulieu, 2020). Assim, faz-se imprescindível salientar que os sistemas relacionais são a “pedra-mestra” nesse tipo de operação (Codd, 1970), por se consolidarem como arquiteturas que utilizam linguagens mais

inteligíveis (Chamberlin, 2012), permitindo que a complexidade da álgebra de dados seja traduzida em comandos intuitivos para o usuário final (Atwood, 2007; Beaulieu, 2020)

O SQL (Structured Query Language) é a linguagem padrão de consulta a bancos de dados relacionais. Sua relação com a Teoria dos Conjuntos é mediada pela Álgebra Relacional: cada comando SQL implementa uma operação de conjuntos sobre relações. No entanto, Chamberlin, um dos criadores do SQL, documenta que a linguagem foi projetada para ser acessível a usuários não matemáticos, fazendo com que o SQL seja operacionalmente acessível, mas tornou menos explícita a semântica formal que determina o comportamento de cada operação. Beaulieu, exemplifica esse distanciamento ao apresentar, no livro “Learning SQL, 2nd edition” consultas SQL como construções sintáticas sem explicitar que são operações de conjuntos com propriedades matemáticas.

O resultado é evidente: profissionais de banco de dados executam operações cujas propriedades matemáticas não dominam completamente, operando sem o ferramental teórico que lhes permitiria raciocinar de forma plena sobre os resultados de suas consultas, originando, conseqüentemente, inconsistências. Sob um prisma paradoxal, poucos investimentos em estruturas de análise e lacunas na padronização da coleta de registros, corroboram limitações na formulação de soluções estratégicas (IBM, Alice Gomstyn e Alexandra Jonker) para empresas. Nesse diapasão, a existência de anomalias nos dados - redundâncias, inconsistência de dados em arquivos diferentes e informações ambíguas ou mal definidas (Data Smells, 2022) -, frutos da ausência parcial dos fundamentos da Teoria dos Conjuntos e das Formas Normais, durante a etapa de modelagem lógica (IBM, Tom Krantz e Alexandra Jonker), dificulta a realização de consultas precisas e a extração de insights confiáveis (BM, Alice Gomstyn e Alexandra Jonker).

Atwood, demonstrou visualmente que cada tipo de JOIN em SQL corresponde a uma região específica do Diagrama de Venn de duas tabelas. Essa contribuição permanece na literatura técnica como recurso didático sem a formalização matemática que lhe conferiria status de instrumento de verificação. Neste viés, este trabalho busca validar o uso de instrumentos lógicos, como diagramas de Venn/Euler, como um mecanismo que garanta a integridade e a acurácia das informações reveladas por dados, passando a complementar, portanto, as atividades das empresas que utilizam sistemas relacionais. Por serem visualmente inteligíveis, esses diagramas tornam conceitos de Álgebra Relacional mais evidentes para a revisão humana, tanto para um leigo, quanto para um profissional na área, comprovando a necessidade de estudos que aprofundem sobre o uso de diagramas de Venn como ferramenta para compreender a Teoria dos Conjuntos e validar a estrutura lógica de sistemas relacionais.

Revisão Conceitual

2.1 Conjuntos

De acordo com Antonio Alfredo Ferreira Loureiro, a Teoria dos Conjuntos é o ramo da matemática que estuda as propriedades e as relações entre coleções de objetos. Um conjunto é entendido como uma coleção de objetos bem definidos, que podem ser qualquer coisa: números, letras, pessoas ou até outros conjuntos e são denominados elementos.

A relação de pertencimento é o conceito fundamental dessa teoria. Segundo Jaime Campos Ferreira, se um objeto x é um dos elementos que compõem o conjunto A , dizemos que x pertence a A , utilizando o símbolo $\in$ ($x \in A$). Caso o objeto não faça parte da coleção, utiliza-se o símbolo $\notin$ ($x \notin A$).

Ferreira reforça que a relação de pertencimento é a base para definir outras relações, como a de inclusão (quando um conjunto está contido em outro).

Na prática, se definirmos um conjunto A como os números pares menores que 10, temos $A = \{2, 4, 6, 8\}$.

- Dizemos que $2 \in A$ (2 pertence a A), pois é um elemento do conjunto
- Dizemos que $3 \notin A$ (3 não pertence a A), pois não é um elemento do conjunto

Essa distinção clara entre o que está "dentro" e o que está "fora" do conjunto, conforme as regras de lógica matemática detalhadas por Ferreira, é o que permite operações como a seleção, a filtragem e a validação de dados em bancos de dados.

A necessidade de definir conjuntos e pertencimento reside na fundamentação de toda a matemática moderna. Joan Bagaria explica que a Teoria dos Conjuntos serve como a linguagem comum e a fundação sobre a qual quase todos os objetos matemáticos (como números, funções e estruturas geométricas) são formalmente definidos. Além disso, Loureiro destaca que essa estrutura permite a organização de informações e a visualização de relações complexas através de métodos gráficos, como os Diagramas de Venn, facilitando a compreensão de interseções e uniões entre diferentes grupos.

A definição moderna de conjunto vai além da simples intuição. Como aponta Bagaria, para evitar contradições lógicas, a matemática utiliza axiomas (regras fundamentais). Um dos mais importantes é o Axioma da Extensionalidade, que afirma que um conjunto é determinado unicamente pelos seus elementos, ou seja, a ordem em que os elementos são apresentados ou a repetição deles não altera a identidade do conjunto.

2.2 Diagramas de Venn

Venn (1880), propõe os diagramas como um instrumento de verificação lógica das relações entre conjuntos, permitindo testar a validade de proposições de forma sistemática e rigorosa.

No escopo do uso de diagramas como instrumentos de verificação das relações entre conjuntos, faz-se imprescindível salientar a diferença entre Diagramas de Venn e Diagramas de Euler. Moktefi e Lemanski (2022) formalizam a propriedade que os distingue: os Diagramas de Venn representam todas as regiões possíveis resultantes da combinação de conjuntos, inclusive as regiões vazias, as quais são marcadas por sombreamento. Por sua vez, os Diagramas de Euler representam apenas as relações efetivamente existentes. Um diagrama que não representa áreas sem elementos não permite verificar se uma operação retorna um conjunto vazio, o que é uma das verificações necessárias para avaliar a precisão dos resultados em consultas SQL. A título de exemplificação, um Diagrama de Venn de dois conjuntos, A e B , possui exatamente quatro regiões: $A \setminus B$ (exclusiva de A), $A \cap B$ (termos em comum ou superposição), $B \setminus A$ (exclusiva de B) e $(A \cup B)^c$ (o complemento da união entre A e B , ou seja, região externa aos conjuntos A e B). Cada

operação fundamental corresponde a uma combinação específica dessas regiões, o que torna o diagrama um mapa completo das possíveis relações entre os dois conjuntos.

2.3 Álgebra Relacional

O Modelo Relacional, proposto por Edgar F. Codd em 1970 conecta os fundamentos da Teoria dos Conjuntos ao domínio computacional de dados. Na seção 1.1 do artigo, Codd define: dada uma coleção de domínios $D_1, D_2, \dots, D_n$, uma relação R sobre esses domínios é um subconjunto do produto cartesiano $D_1 \times D_2 \times \dots \times D_n$. Nesse modelo, cada relação é composta por atributos (colunas) e tuplas (linhas), cujos valores pertencem a domínios específicos. Essa é a mesma definição matemática de relação formulada por Enderton (1977). Na seção 2.1, Codd define as operações relacionais, como seleção, projeção, união, interseção e diferença, como operações de conjuntos sobre os subconjuntos do produto cartesiano. Dentre essas operações, a seleção é responsável pelo filtro de tuplas (linhas), enquanto a projeção seleciona atributos específicos. A título de exemplificação, em um conjunto de dados contendo os atributos nome e nota, a seleção poderia ser utilizada para retornar apenas as tuplas cujas notas sejam superiores a cinco. Em contrapartida, a projeção selecionaria apenas atributos específicos, como a coluna correspondente aos nomes. Sob esse prisma, evidencia-se que o Modelo Relacional consolida a relação entre a Teoria de Conjuntos e o domínio de dados.

Sob o diapasão das inovações que Codd introduziu, a exigência de que a união de duas relações só é definida se elas possuírem o mesmo número de atributos com domínios correspondentes compatíveis, pois apenas assim o resultado constitui uma relação válida com esquema consistente, uma decisão de modelagem, não uma propriedade da Teoria dos Conjuntos, e demonstra que a transposição de Codd foi deliberada e tecnicamente informada. Stonebraker e Hellerstein (2005) documentam que a vantagem do Modelo Relacional sobre os sistemas hierárquicos se deveu à sua base conjuntista rigorosa. Sistemas que careciam de fundamentos matemáticos bem definidos não conseguiam garantir que a mesma consulta produzisse sempre o mesmo resultado, nem que operações equivalentes fossem reconhecidas como idênticas. Nesse contexto, a base conjuntista resolveu esses problemas, tornando as operações sobre dados baseadas na Teoria dos Conjuntos, tornando-as previsíveis e independentes de implementação.

Silberschatz, Korth e Sudarshan (2019) descrevem a Álgebra Relacional como extensão da Teoria dos Conjuntos e estabelecem a correspondência entre operações relacionais e SQL. Date (2004) descreve cada operação da Álgebra Relacional como operação de conjuntos, incluindo a união, interseção, diferença, produto cartesiano, seleção e projeção. Date também demonstra que a precisão de consultas SQL exige o domínio da teoria relacional e, por extensão, das operações de conjuntos que a fundamentam. Heuser (2009) identifica que erros recorrentes em projetos de banco de dados têm origem na incompreensão das relações lógicas entre os conjuntos de dados que as tabelas representam.

2.4 Banco de Dados

Um banco de dados é uma coleção organizada de dados persistentes que atende a múltiplos usuários e aplicações. O modelo de banco de dados relacional fundamenta-se na formalização proposta por Codd, segundo a qual os dados devem ser organizados em relações compostas por tuplas e atributos (Codd, 1970), ou seja, no modelo relacional, os dados são organizados em tabelas com esquema definido, sendo pouco flexível, mas muito

organizado, íntegro e de fácil consulta, infere-se, portanto, que sua estrutura não é aleatória, mas planejada de acordo com as necessidades de armazenamento, organização e recuperação das informações. Nesse sentido, em um banco de dados relacional, uma tabela, por exemplo, não constitui apenas um mecanismo prático de armazenamento, mas uma representação formal de um conjunto matemático, em que cada tupla corresponde a um elemento pertencente a um domínio definido. A adoção desse modelo está associada à necessidade de reduzir redundâncias e inconsistências, problemas recorrentes em sistemas que não utilizam uma estrutura lógica rigorosa (Heuser, Projeto de Banco de Dados). Conforme observado por Rahm e Do, a ausência de organização adequada dos dados implica em custos elevados de correção e manutenção, comprometendo a confiabilidade das informações (Rahm & Do, Data Cleaning). Do ponto de vista teórico, o modelo relacional é uma aplicação direta da teoria dos conjuntos e da lógica de predicados, sendo essas bases essenciais para a consistência estrutural dos dados (Date, SQL and Relational Theory). A operacionalização desses conceitos ocorre por meio da linguagem SQL, desenvolvida no contexto do System R, que traduz operações matemáticas em comandos declarativos acessíveis ao usuário (Chamberlin, Early History of SQL).

1º - Matemática e Diagramas de Venn:

Um cadastro de clientes de uma empresa pode ser considerado um banco de dados relacional. Nele, uma coluna indica o código do cliente, outra indica o seu nome e uma terceira indica sua idade, por exemplo. Por mais simples que essa estrutura de dados relacional possa ser, para que não haja irregularidade nos dados contidos nela, diversos axiomas devem ser respeitados durante seu preenchimento.

No campo da matemática, muito antes de existirem bancos de dados como o MySQL ou a Cassandra, a Teoria dos Conjuntos era o ramo responsável por estudar as propriedades e as relações entre coleções de objetos. Nesse escopo, um conjunto é entendido como uma coleção de objetos bem definidos, que podem ser qualquer coisa: números, letras, pessoas ou até outros conjuntos, e são denominados elementos (LOUREIRO, Antonio Alfredo Ferreira). Tomando como exemplo a situação discutida previamente:

$$\text{Clientes} \subseteq \text{dom}(ID), \text{dom}(\text{Nome}), \text{dom}(\text{Idade})$$

$$\text{dom}(ID) = \mathbb{Z}^+$$

$$\text{dom}(\text{Nome}) = \text{String}$$

$$\text{dom}(\text{Idade}) = \mathbb{Z}^+$$

ID (PK)	Nome	Idade
1	Ana	16
2	Bruno	23
3	Carlos	50

Polindo ainda mais o exemplo anterior, suponha que possuamos duas tabelas de clientes que contenham os dados da mesma empresa, porém em filiais diferentes, consequentemente contendo dados de clientes diferentes.

ID (PK)	Nome	Idade
4	Antunes	7
2	Bruno	23
5	Cecilia	84

Caso queiramos juntar ambas as tabelas, a fim de criar uma base de dados unificada, podemos utilizar a operação de UNIÃO, criando uma tabela nova que contém os dados das duas tabelas originais.

$$A \cup B = \{x \mid x \in A \vee x \in B\} \quad (1)$$

ID (PK)	Nome	Idade
1	Ana	16
2	Bruno	23
3	Carlos	50
4	Antunes	7
5	Cecilia	84

Vale ressaltar que um fenômeno fulcral ocorre durante essa operação: a exclusão de duplicatas. Note que em ambas as tabelas existe um mesmo cadastro de um cliente. Durante a união, elementos repetidos são tratados como uma única ocorrência. Esta é a primeira regra da Álgebra Relacional (baseada na Teoria dos Conjuntos) apresentada neste artigo, e o seu impacto torna-se evidente durante a modelagem de bancos de dados mais avançados.

Agora, suponha que as duas tabelas que desejamos unir sejam essas:

ID (PK)	Nome	Idade
1	Ana	16
2	Bruno	23
3	Carlos	50

ID (PK)	Nome	Idade
1	Antunes	7
2	Bruno	23
3	Cecilia	84

Isso resultaria em um erro clássico, baseado em um dos axiomas da Álgebra Relacional, fundamentada na Teoria dos Conjuntos, que diz que, por mais que o domínio das relações seja compatível, a existência de pessoas distintas com o mesmo ID configura uma infração do axioma da Chave Primária.

A Chave Primária consiste em um ou vários atributos cujo valor deve ser obrigatoriamente único para cada tupla de uma relação, atuando como um identificador que impede a duplicata de dados e garante a integridade referencial do modelo (HEUSER, 2009).

Conclui-se, portanto, que a modelagem bem-sucedida de bancos de dados relacionais, por mais simples que seja, exige o conhecimento matemático acerca da Teoria dos Conjuntos e suas propriedades e pode ser simplificada por meio da ilustração de um diagrama de Venn.

2° - Álgebra Relacional:

O Modelo Relacional, proposto por Edgar F. Codd, conecta os fundamentos da Teoria dos Conjuntos ao domínio computacional de dados (CODD, 1970). Nesse modelo, cada relação é composta por atributos (colunas) e tuplas (linhas), cujos valores pertencem a domínios específicos. Antes de aplicar, na prática, a Teoria dos Conjuntos no contexto computacional, ou seja, a Álgebra Relacional, faz-se imprescindível compreender as estruturas de dados comuns que empresas utilizam para armazenar informações importantes.

Uma tabela, como mostrada na figura X, representa logicamente uma relação R , na qual R é um subconjunto do produto cartesiano entre os domínios dos atributos (conjunto de valores válidos que podem ser armazenados em um atributo).

Visualmente, cada atributo é representado por uma coluna cujo nome também é apresentado no cabeçalho (HEUSER, 2009).

Com isso em mente, no mercado de trabalho, hodiernamente funcionários utilizam operações da Álgebra Relacional, a fim de obter uma melhor visualização dos dados. No cenário apresentado previamente, podemos aplicar filtros que mostrem apenas pessoas maiores de idade, ou selecionar, para impressão, apenas o nome e a idade das pessoas. Ambas as operações são fundamentadas em conceitos da Álgebra Relacional, sendo elas, respectivamente, a Seleção - que filtra tuplas com base em um predicado booleano e se define formalmente como:

$$\sigma_{\phi}(R) = \{ t \mid t \in R \wedge \phi(t) \} \quad (2)$$

Onde R é a relação original, t é uma tupla (linha da tabela original) que pertença a R e que satisfaça a condição imposta - e a Projeção - operação que isola um subconjunto de atributos. Define-se formalmente como:

$$\pi_L(R) = \{ t[L] \mid t \in R \} \quad (3)$$

Onde L são os atributos (colunas) que queremos selecionar da relação R original e $t[L]$ é a tupla resultante dos atributos selecionados (HEUSER, 2009). Codd define essas e outras operações relacionais, como projeção, união, interseção e diferença, como operações de conjuntos sobre os subconjuntos do produto cartesiano (CODD, 1970), evidenciando que o Modelo Relacional consolida a relação entre a Teoria dos Conjuntos e o domínio de dados no cenário computacional. Além disso, na tentativa de padronizar o modelo relacional, Codd introduziu exigências para o modelo que, para um leigo, possam parecer irrelevantes, mas que servem como pedra-mestra que confere um funcionamento regular ao banco de dados relacional (DATE, 2004). Observe abaixo as exigências e os problemas gerados pela negligência de tais decisões:

1. Compatibilidade de Esquemas na União: A exigência é de que a união de duas relações só é definida se elas possuírem o mesmo número de atributos com domínios correspondentes compatíveis, pois apenas assim o resultado constitui uma relação válida com esquema consistente (ELMASRI; NAVATHE, 2011). Essa restrição configura uma decisão de modelagem, e não uma propriedade da Teoria dos Conjuntos (DATE, 2004).

PK	Nome	Idade	Escola
1	Luis	15	A
2	Carlos	9	B
3	Amanda	12	C

PK	Nome	Idade	Alergias
4	Antonio	22	Castanhas
5	Breno	14	Cachorro
6	Carlos	61	Medicamentos

PK	Nome	Idade	Alergias	Escola
1	Luis	15		A
2	Carlos	9		B
3	Amanda	12		C
4	Antonio	22	Castanhas	
5	Breno	14	Cachorro	
6	Carlos	61	Medicamentos	

2. Restrição de Integridade de Entidade: Em um sistema relacional, diferentemente do estipulado na Teoria dos Conjuntos, não é possível introduzir $t_i = \{1, Ana, 7\}$ à tabela abaixo, visto que um único valor de chave primária (valor 1) mapearia duas instâncias distintas no mesmo corpo de dados (HEUSER, 2009).

PK	Nome	Idade
1	Carlos	12
2	João	9
3	Amanda	12

PK	Nome	Idade
1	Amanda	7
1	Carlos	12
2	João	9
3	Amanda	12

Existem normalizações que previnem outros tipos de anomalias (HEUSER, 2009). Estas serão abordadas futuramente. Stonebraker e Hellerstein (2005), em *What Goes Around Comes Around*, documentam que a vantagem do Modelo

Relacional sobre os demais se deveu à sua rigorosa base conjuntista. Sistemas que careciam de fundamentos matemáticos bem definidos não conseguiam garantir que a mesma consulta produzisse sempre o mesmo resultado, nem que operações equivalentes fossem reconhecidas como tais. Além disso, Heuser (2009), em *Projeto de Banco de Dados* (6ª ed.), identifica que erros recorrentes em projetos de banco de dados têm origem na incompreensão das relações lógicas entre os conjuntos de dados das tabelas. Sob essa perspectiva, conclui-se que o conhecimento teórico, por mais críptico que seja, é fulcral para construir bases de dados confiáveis. Isso, conforme destacado previamente, é essencial para garantir a extração de informações valiosas, o que aumenta a competitividade das empresas no mercado hodierno -atual-. EM DÚVIDA SE PRECISA OU NÃO DE REFERÊNCIA

3° - Formas Normais

O próximo passo para obter uma maestria sob o uso eficiente de dados é conhecer as Formas Normais, que são normatizações que garantem a ausência de anomalias de dados e otimizam a funcionalidade de um banco de dados. Antes de conceituar as 3 Formas Normais (note que, em algumas literaturas são expostas 5 ou mais Formas, mas as 3 primeiras já são muito boas para evitar a grande maioria de imbróglis), precisamos conceituar os tipos de dependência em um banco de dados e as categorias de atributos que causam problemas no nosso modelo.

A princípio, vamos discutir sobre as dependências encontradas em um banco de dados. Tomando como exemplo a tabela a seguir, é possível identificar 3 chaves primárias (ID_Aluno, ID_Materia e Semestre) uma vez que para acessar a nota, por exemplo, apenas o ID_Aluno não é suficiente para obter o resultado, ou seja, precisamos do valor de outros 3 atributos para descobrir a minha informação. Este

ID_ Aluno	ID_ Matéria	Materia	Semestre	Nota
222	1	Cálculo	1	8
222	2	Física	1	5
222	3	Álgebra	1	6
222	4	Química	1	10

tipo de situação é conhecido como Dependência Funcional Parcial - quando meu atributo não chave é identificável por meio de parte da minha chave primária-, uma vez que as notas, nesta tabela, não dependem do ID_Aluno, mas sim do Semestre e ID_Materia.

Neste mesmo eixo, temos um caso de Dependência Funcional Transitiva quando um elemento é acessado por meio de um campo que não é a chave primária. Tendo como referência a tabela abaixo, vemos que o a coluna 'Funcao' não depende do ID_User (nossa chave primária), mas sim do 'ID_Funcao', logo a coluna 'Funcao' é redundante, uma vez que eu poderia remover ela e o banco de dados funcionaria da mesma forma, exigindo apenas a formalização de que o 'ID_Funcao' 1 representa Aluno, e assim por diante.

ID_User	ID_Funcao	Funcao	Salário	Nome
1	1	Aluno	-1500	Auris
2	2	Professor	3000	Bento
3	1	Aluno	-1500	Cecilia
4	3	Diretor	7000	Alexandre

O último conceito que é preciso dominar antes de compreender as Formas Normais são os Atributos multivalorados e os Atributos compostos. Como última tabela analisada nesta seção, o registro abaixo conta com ambos os tipos de atributos, veja:

CPF	Nome	Funcao	Telefone	Endereço
111	Marco	Aluno	(61) 99627-3615 (61) 99372-2910	SQS 305, Bloco J Brasília, DF
222	Gael	Professor	(11) 98888-1111	Chácara 12, Conjunto B Brotas, SP
333	Davi	Aluno	(21) 97777-2222 (21) 97777-3333 (21) 97777-4444	Avenida Atlântica, Rio de Janeiro
444	Rafael	Professor	(31) 96666-5555	Rua da Bahia 10 Belo Horizonte MG

Neste caso, observamos a ocorrência de Atributos Multivalorados nos números de telefone, uma vez que o registro de PK 111 e 333 possuem mais de um valor no atributo telefone, configurando uma possível inconsistência no nosso modelo. Além disso, também ocorrem Atributos Compostos no atributo endereço, já que está sendo guardado a rua, a cidade e o estado do usuário, dados distintos no mesmo atributo

Após discorrer acerca das anomalias e redundâncias frequentes em bancos de dados relacionais, faz-se imprescindível avaliar e conhecer as soluções para estes obstáculos para uma análise precisa de dados. As Formas Normais são processos compostos por procedimentos e boas práticas aplicadas para garantir que um banco de dados esteja bem estruturado. O objetivo central é evitar anomalias, originadas pela má estruturação da coleta e armazenamento de dados. Utilizando as Formas nos exemplos anteriores podemos resolver todos os problemas:

A Primeira Forma Normal é específica para resolver problemas encontrados na tabela (x). Deve-se seguir os seguintes passos:

1. Reconhecer o atributo multivalorado, no caso da tabela (x), é a coluna "Telefone"
2. Criar uma nova tabela com uma chave, no caso da tabela (x), é o CPF, e uma linha para cada atributo multivalorado
3. Excluir da tabela original a coluna multivalorada

No final, sobram as seguintes tabelas:

CPF	Nome	Funcao	Endereço
111	Marco	Aluno	SQS 305, Bloco J Brasília, DF
222	Gael	Professor	Chácara 12, Conjunto B Brotas, SP
333	Davi	Aluno	Avenida Atlântica, nº 1702 Rio de Janeiro, RJ
444	Rafael	Professor	Rua da Bahia 10 Belo Horizonte MG

CPF	Telefone
111	(61) 99627-3615
111	(61) 99372-2910
222	(11) 98888-1111
333	(21) 97777-4444
333	(21) 97777-3333
333	(21) 97777-2222
444	(31) 96666-5555

A Segunda Forma Normal

A Terceira Forma Normal

4° - Restauração de Banco de dados

Nesta seção, será utilizado o seguinte banco de dados:

	CPF	Nome	D_FUNCAO	Funcao	Salario	Telefone	Endereço
0	46547	Gabriel Sc	3	Senior	7000	96203163	Av B, Bloco O - SP
1	19010	Leonardo	3	Senior	7000	98111285	Av B, Bloco K - PA
2	26430	Amanda C	1	Junior	2000	96062028	Av B, Bloco K - MG
3	23713	Pedro Lim	1	Junior	2000	92048266	Alameda E, Bloco A - RJ
4	43857	Lucas Car	2	Plenior	4500	92603343	Alameda E, Bloco K - DF
5	26379	Helena Sil	1	Junior	2000	98085741	Rua A, Bloco J - SP
6	38414	Amanda V	0	Estagiario	1500	94755368	Alameda E, Bloco L - SP
7	29927	Larissa Ma	3	Senior	7000	92122882	Alameda E, Bloco J - DF
8	16024	Gabriel M	2	Plenior	4500	94449591	Av B, Bloco L - MG
9	28049	Ricardo Sc	1	Junior	2000	93899536	Rua A, Bloco K - MG
10	13899	Beatriz Fr	0	Estagiario	1500	97987468	Rua A, Bloco O - MG
11	40744	Camila Sil	2	Plenior	4500	98087441	Av B, Bloco L - PA
12	34228	Lucas Soa	2	Plenior	4500	93464242	Av B, Bloco J - PA
13	22160	Lucas Pere	1	Junior	2000	99746869	Rua A, Bloco J - DF
14	20080	Rodrigo S	0	Estagiario	1500	97798415	Alameda D, Bloco J - RJ
15	12191	Larissa Ma	2	Plenior	4500	93017745	Av B, Bloco J - DF

*Amostra obtida utilizando o comando head(15) de um conjunto de dados fictícios com 1000 registros

Atente-se ao fato de que temos casos de atributos multivalorados e compostos e dependências parciais e transitivas. Veja como polir o banco de dados fictícios por meio da aplicação das Formas Normais, que são fundamentadas nos conceitos abordados na seção 3.2 Álgebra Relacional do projeto:

Aplicação da 1ª Forma Normal

No dataSet exemplo, a fim de remover as inconsistências e anomalias, deve-se começar impregnado a primeira Forma Normal. Utilizando o comando *head*, somos apresentados as primeiras 5 linhas do nosso banco de dados, nela é possível identificar a existência de atributos compostos na coluna Endereço, que contém informações de avenida/rua, bloco e estado. A FN1 corrige este imbróglio da seguinte forma: Cada informação deve ser salva em um atributo específico. No caso apresentado, iremos adicionar ao banco de dados 3 colunas, 'Rua', 'Bloco' e 'Estado', e simultaneamente, vamos remover a coluna 'Endereço'.

CPF	Nome	ID_FUNCAO	Funcao	Salario	Telefone	Endereço
46547	Gabriel S	3	Senior	7000	962031635	Av B, Bloco O - SP
19010	Leonardo	3	Senior	7000	981112855	Av B, Bloco K - PA
26430	Amanda	1	Junior	2000	960620280	Av B, Bloco K - MG
23713	Pedro Li	1	Junior	2000	920482660	Alameda E, Bloco A - RJ
43857	Lucas Ca	2	Plenior	4500	926033435	Alameda E, Bloco K - DF

**Amostra obtida por meio do head() em python. Note que a chave primária padrão foi cortada do exemplo para dar ênfase no imbróglio, mas ela ainda existe*

No final, vamos ter o seguinte resultado:

CPF	Nome	D_FUNCAO	Funcao	Salario	Telefone	Rua	Bloco	Estado
46547	Gabriel S	3	Senior	7000	962031635	Av B	Bloco O	SP
19010	Leonardo	3	Senior	7000	981112855	Av B	Bloco K	PA
26430	Amanda	1	Junior	2000	960620280	Av B	Bloco K	MG
23713	Pedro Li	1	Junior	2000	920482660	Alameda E	Bloco A	RJ
43857	Lucas Ca	2	Plenior	4500	926033435	Alameda E	Bloco K	DF

**Amostra obtida por meio do head() em python. Note que a chave primária padrão foi cortada do exemplo para dar ênfase no imbróglio, mas ela ainda existe*

A segunda anomalia corrigida pela Primeira Forma Normal é a existência de atributos multivalorados. Estes são mais de um dado do mesmo tipo em uma coluna. Isso é uma redundância e devemos seguir os seguintes passos para consertar o banco de dados:

- 1) Reconhecer qual coluna possui atributos multivalorados
- 2) Criar uma nova tabela que guarde, em uma tupla, a chave primária da tabela original e o atributo multivalorado em cada tupla distinta
- 3) Remover o atributo multivalorado da tabela original

Observe o esquema a seguir:

M	N	O	P	Q	R	S	T	U
CPF	Nome	D_FUNCAO	Funcao	Salario	Telefone	Rua	Bloco	Estado
29927	Larissa M	3	Senior	7000	97358522 94168781	Alameda	J	DF
49814	Matheus	3	Senior	7000	97204892 93181292	Rua A	A	MG
46690	Bruno Fre	2	Plenior	4500	92478718 98582372	Alameda	O	PA
36371	Gabriel A	3	Senior	7000	92729060 93062842	Alameda	L	MG
37641	Bárbara S	1	Junior	2000	93509707 93956952	Alameda	L	MG

O atributo Telefone contém mais de 1 registro, em alguns casos. Depois da aplicação da 1ª Forma Normal, vamos receber as seguintes tabelas:

W	X
CPF	Telefone
29927	97358522
29927	94168781
49814	97204892
49814	93181292
46690	92478718
46690	98582372
36371	92729060
36371	93062842
37641	93509707
37641	93956952

A	B	C	D	E	F	G	H
CPF	Nome	ID_FUNCAO	Funcao	Salario	Rua	Bloco	Estado
29927	Larissa Martins	3	Senior	7000	Alameda E	J	DF
49814	Matheus Dias	3	Senior	7000	Rua A	A	MG
46690	Bruno Freitas	2	Plenior	4500	Alameda E	O	PA
36371	Gabriel Andrade	3	Senior	7000	Alameda E	L	MG
37641	Bárbara Santos	1	Junior	2000	Alameda C	L	MG

Aplicação da 2° Forma Normal

Aplicação da 3° Forma Normal

5° - SQL

O MySQL é um Sistema de Gerenciamento de Banco de Dados Relacional (SGBDR), ferramenta utilizada para realizar consultas, incluir e modificar dados estruturados. Os SGBDs são essenciais na computação moderna por permitirem níveis granulares de acesso e privilégios aos dados, ou seja, restrições

acerca de quem pode ou não pode ver e editar certos dados, garantindo um controle seguro e eficiente sobre grandes volumes de informações. Atualmente, o MySQL destaca-se como um dos SGBDs mais difundidos no mercado devido à sua natureza de código *open-source*, gratuidade e suporte a diversas linguagens de programação. Além disso, o MySQL trabalha com estruturas de dados relacionais, garantindo maior integridade - a sigla ACID (Atomicidade, Consistência, Isolamento, Durabilidade) descreve os pilares dessa estrutura - e facilidade na execução de consultas.

No que tange à temática deste artigo, é válido ilustrar a forma com que o MySQL executa suas funções de manipulação de dados, para que, conseqüentemente, seja visível a participação ativa de diagramas de Venn para auxiliar na validação lógica dos comandos. A análise a seguir demonstrará como os comandos lógicos do software são reflexos diretos dos operadores da Álgebra Relacional e da Teoria dos Conjuntos, tornando explícita a viabilidade dos diagramas de Venn como ferramentas de validação lógica dessas operações.

1) WHERE

No MySQL, o comando WHERE implementa a operação de seleção, sendo esta um pilar essencial para filtrar as tuplas de uma tabela para que apenas registros que atendam uma condição específica sejam exibidos. Na tabela(x), caso o desenvolvedor queira ver a média salarial dos registros sediados no estado de SP, podemos executar o seguinte comando “SELECT * FROM tabela(incluir numero) WHERE estado = ‘SP’;”, que vai exibir apenas os registros cujo o atributo “Estado” esteja preenchido com “SP”, sendo este o primeiro passo para a execução da tarefa. Este comando é representado na teoria dos conjuntos por:

$$x = \{t \in \text{tabela}(x) / t = \text{'SP'}\}$$

O uso de diagramas de Venn é uma ferramenta que facilita a visualização deste tipo de consulta, pois permite que você, como usuário ou programador, observe, de forma mais palpável, a fórmula (numero da fórmula citada acima):

(*ADICIONAR DIAGRAMA DE VENN*)

2) JOIN

3)) SELECT

Por fim, o comando SELECT implementa a operação de projeção, ou seja, é uma operação que filtra a tabela verticalmente, selecionando apenas os atributos desejados. Observe que, na tabela (x), o comando “**SELECT** cpf, salario **FROM** tabela(colocar número da tabela corrigida);” pode ser reescrito, de forma críptica, para alguém que não domina totalmente teoria dos conjuntos, como:

$$x = \{t_{\text{cpf}}, t_{\text{salario}} / t \in \text{tabela}(x)\}$$

Atente-se que, dentre todas as consultas exemplificadas neste bloco, o SELECT é o único que não tem uma correspondência em um Diagrama de Venn, uma vez que o select altera a dimensão da tabela exibida, tornando incompatível para o diagrama de Venn, cujo propósito é representar a relação de pertencimento à um conjunto (conceito melhor explicado na seção x do trabalho)

Referências Bibliográficas (Ordem Alfabética Geral)

ATWOOD, Jeff. A Visual Explanation of SQL Joins. **Coding Horror**, 2007. Disponível em: <https://blog.codinghorror.com/a-visual-explanation-of-sql-joins/>. Acesso em: 16 maio 2026.

BAGARIA, Joan. Set Theory. *In*: ZALTA, Edward N. (ed.). **The Stanford Encyclopedia of Philosophy**. Stanford: Stanford University, 2023. Disponível em: <https://plato.stanford.edu/entries/set-theory/>. Acesso em: 16 maio 2026.

BEAULIEU, Alan. **Learning SQL: Generate, Manipulate, and Retrieve Data**. 3. ed. Sebastopol: O'Reilly Media, 2020.

CHAMBERLIN, Donald D. Early History of SQL. **IEEE Annals of the History of Computing**, v. 34, n. 4, p. 78-82, out./dez. 2012.

CODD, E. F. A Relational Model of Data for Large Shared Data Banks. **Communications of the ACM**, v. 13, n. 6, p. 377–387, 1970.

DATE, C. J. **Introdução a Sistemas de Bancos de Dados**. Rio de Janeiro: Elsevier, 2004.

DATE, C. J. **SQL e Teoria Relacional**: como escrever código SQL preciso. Tradução de Edson Furmankiewicz. São Paulo: Novatec, 2012.

FERREIRA, Jaime Campos. **Elementos de lógica matemática e teoria dos conjuntos**. Lisboa: Departamento de Matemática do Instituto Superior Técnico, 2001. Disponível em:
<https://www.ime.usp.br/mat/206/textos/elmtcJaimeCamposFerreira.pdf>. Acesso em: 18 maio 2026.

FLAMMA DESIGN. **O que é inconsistência de dados?**. [S. l.]: Flamma Design, [20--]. Glossário. Disponível em:
<https://flammadeesign.com.br/glossario/o-que-e-inconsistencia-de-dados/>. Acesso em: 18 maio 2026.

HEUSER, Carlos Alberto. **Projeto de Banco de Dados**. 6. ed. Porto Alegre: Bookman, 2009.

IBM. **O que é a redundância de dados?**. [S. l.]: IBM Think, 2024. Disponível em:
<https://www.ibm.com/br-pt/think/topics/data-redundancy>. Acesso em: 18 maio 2026.

IBM. **O que é normalização de banco de dados?**. [S. l.]: IBM Think, 2024. Disponível em: <https://www.ibm.com/br-pt/think/topics/database-normalization>. Acesso em: 18 maio 2026.

KENT, William. A Simple Guide to Five Normal Forms in Relational Database Theory. **Communications of the ACM**, New York, v. 26, n. 2, p. 120-125, fev. 1983.

LOUREIRO, Antonio Alfredo Ferreira. **Teoria dos Conjuntos**. Belo Horizonte: UFMG, [20--]. Disponível em:
https://homepages.dcc.ufmg.br/~loureiro/md/md_5TeoriaDosConjuntos.pdf. Acesso em: 16 maio 2026.

RAHM, Erhard; DO, Hong Hai. Data Cleaning: Problems and Current Approaches. **IEEE Bulletin of the Technical Committee on Data Engineering**, [s. l.], v. 23, n. 4, p. 3-13, 2000.

SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. **Query Optimization**. [S. l.: s. n.], [20--]. (Slides de apoio ao livro Database System Concepts). Disponível em: <https://www.db-book.com/slides-dir/PDF-dir/ch16.pdf>. Acesso em: 16 maio 2026.

STONEBRAKER, Michael; HELLERSTEIN, Joseph M. What Goes Around Comes Around. *In: Readings in Database Systems*. 4. ed. [S. l.]: MIT Press, 2005.

WINAND, Markus. **SQL Performance Explained**: everything developers need to know about SQL performance. [S. l.]: Markus Winand, 2012.

Principais artigos

- ATE, C. J. SQL e Teoria Relacional: como escrever código SQL preciso. (banco de dados)
- DATE, C. J. **Introdução a Sistemas de Bancos de Dados**. (banco de dados e formas normais)
- FERREIRA, Jaime Campos. Elementos de lógica matemática e teoria dos conjuntos. **Lisboa: Departamento de Matemática do Instituto Superior Técnico**, 2001. (teoria dos conjuntos)
- Codd, E. F. A Relational Model of Data for Large Shared Data Banks. *Communications of the ACM*, v. 13, n. 6, p. 377–387, jun. 1970. DOI: 10.1145/362384.362685. Prêmio Turing 1981. Seção 1.1: define relação como subconjunto do produto cartesiano. Seção 2.1: define

operações relacionais como operações de conjuntos. Elo formal entre Teoria dos Conjuntos e Álgebra Relacional.

- (álgebra relacional)